KULLIYYAH OF ENGINEERING (KOE)

MECHATRONICS SYSTEM INTEGRATION (MCTA3203)

SEMESTER 1, 25/26

SECTION 1

PROJECT REPORT WEEK 9

TITLE: Image/Video input interfacing with microcontroller

PREPARED BY :

NO	NAME	MATRIC NO
1	MUHAMMAD NAUFAL HAKIMI BIN IRWAN AFFANDI	2313041
2	MUHAMAD IQBAL BIN MD ISA	2313247
3	ADAM NABIL HANIFF BIN RUZAIMI	2313203

DATE OF SUBMISSION: _/_/2025

ABSTRACT

This experiment focuses on implementing a color detection system utilizing an Arduino Uno and the Gravity HuskyLens AI Vision Camera. The objective was to design, set up, and test a system capable of detecting three different learned colored objects. The experiment included testing the system with the learned colors and observing the detection accuracy and the display of bounding boxes and color labels on the HuskyLens. The results were monitored through the Arduino Serial Monitor. The subsequent analysis will evaluate the accuracy and consistency of color detection, consider the impact of lighting conditions, and assess the system's performance, allowing for identification of challenges and proposals for future improvements.

TABLE OF CONTENT

1.0 INTRODUCTION **2**
2.0 MATERIALS AND EQUIPMENT **2**
3.0 EXPERIMENTAL SETUP **3**
4.0 METHODOLOGY **3**
5.0 DATA COLLECTION **6**
6.0 DATA ANALYSIS **7**
7.0 RESULTS **7**
8.0 DISCUSSION **8**
9.0 CONCLUSION **9**
10.0 RECOMMENDATIONS **9**
11.0 REFERENCES **10**
12.0 APPENDICES **10**
13.0 ACKNOWLEDGEMENT **11**
14.0 STUDENTS DECLARATION **11**

1.0 INTRODUCTION

With the integration of AI in embedded systems, vision-based color detection offers enhanced robustness and flexibility. The HuskyLens AI Vision Camera combines color detection, face recognition, and object tracking capabilities, making it a powerful sensor for intelligent robotics applications.

This experiment demonstrates how to use the HuskyLens in color recognition mode with Arduino Uno. The camera is trained to recognize three different colors, and the Arduino program retrieves detection results through serial communication. This setup helps understand the advantages of AI-assisted sensing in terms of detection speed, accuracy, and ease of calibration.

2.0 MATERIALS AND EQUIPMENT

Hardware:

Item	Function
Arduino Uno	The primary control board.
Gravity: HuskyLens AI Vision Camera	An AI sensor that uses **Color Recognition mode** to detect up to three learned colors. Sends the detected color ID and coordinates to the Arduino.
USB Cable for Arduino connection	Used for uploading the sketch (code) and for providing power to the Arduino Uno.
3 Colour RGB LED Module	An output device that can be used to **visually indicate** which color ID the HuskyLens has detected.
Power supply	Ensures the Arduino Uno and connected HuskyLens are powered reliably for operation.
Breadboard	For assembling the physical circuit
Jumper Wire	For making electrical connections on the breadboard

Software:

- Arduino IDE
- *DFRobot_HuskyLens* library
- Serial Monitor (for viewing color ID and coordinates)

3.0 EXPERIMENTAL SETUP

1. Connect HuskyLens to Arduino Uno via UART interface using **SoftwareSerial** on pins D4 (RX) and D5 (TX).
2. Power HuskyLens using Arduino's 5V and GND pins.
3. Switch HuskyLens to **Color Recognition Mode** using onboard buttons.
4. Train the camera by long-pressing the learning button to record three color samples (e.g., red, green, blue).
5. Run the Arduino program to receive detected color IDs and coordinates through serial output.
6. Test the setup using different colored objects and observe the real-time recognition results on the Serial Monitor and HuskyLens display.
7. Add LEDs or actuators controlled based on color detection results.

4.0 METHODOLOGY

Hardware Configuration:
Connect the HuskyLens to the Arduino Uno using the SoftwareSerial interface.

- TX pin (HuskyLens) - Arduino D4
- RX pin (HuskyLens) - Arduino D5

Connect the 3 Colour RGB LED Module to the Arduino Uno:

- Red LED pin - Arduino D9
- Green LED pin - Arduino D10
- Blue LED pin - Arduino D11

Arduino Library Installation:

- Open Arduino IDE and install the *DFRobot_HuskyLens* library from the Library Manager.
- Include the library in the Arduino sketch.

Arduino Programming:

- Initialize communication between Arduino and HuskyLens.
- Use commands to request color detection data and read object ID, position (x, y), and size.
- Display color ID data on the Serial Monitor.
- Checks the detected result.ID and controls the RGB LED accordingly.

Code Used:

```
#include <SoftwareSerial.h>
#include <DFRobot_HuskyLens.h>

SoftwareSerial mySerial(4, 5);  // RX, TX
HUSKYLENS huskylens;

// RGB LED pin definitions
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  Serial.begin(9600);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Turn off all colors initially
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);

  while (!huskylens.begin(mySerial)) {
    Serial.println("HuskyLens not connected!");
    delay(500);
  }

  Serial.println("HuskyLens Ready");
}

void loop() {
  if (!huskylens.request()) {
    Serial.println("Request failed");
    return;
  }

  if (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    Serial.print("ID: ");
```

```
    Serial.print(result.ID);
    Serial.print("  X: ");
    Serial.print(result.xCenter);
    Serial.print("  Y: ");
    Serial.println(result.yCenter);

    // Turn off all LEDs first
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, LOW);
    digitalWrite(bluePin, LOW);

    // Light up based on detected ID
    if (result.ID == 1) {
      digitalWrite(redPin, HIGH);      // RED ON
      Serial.println("→ Showing RED");
    }
    else if (result.ID == 2) {
      digitalWrite(greenPin, HIGH);    // GREEN ON
      Serial.println("→ Showing GREEN");
    }
    else if (result.ID == 3) {
      digitalWrite(bluePin, HIGH);     // BLUE ON
      Serial.println("→ Showing BLUE");
    }

    delay(1000);
  }
}
```

Color Training and Detection:

- Use the HuskyLens onboard interface to enter “Color Recognition” mode.
- Point the camera at an object and long-press the button to save the color.
- Train Color 1 (e.g., Red): Point the camera at a red object and long-press the learning button to save it as Color ID 1.
- Train Color 2 (e.g., Green): Repeat the process with a green object, saving it as Color ID 2.
- Train Color 3 (e.g., Blue): Repeat the process with a blue object, saving it as Color ID 3.

Testing and Observation:

- Place different colored objects in the camera’s view.
- Observe the bounding boxes and labels on the HuskyLens display.
- Verify detection accuracy and speed from Serial Monitor output.

- Open the Serial Monitor (9600 baud) and verify that:
    - The correct **ID** (1, 2, or 3) and coordinates are being displayed.
    - The corresponding color (Red, Green, or Blue) on the **3 Colour RGB LED Module lights up**.

Data Evaluation:

- Compare recognition results against actual colors.
- Analyze response time and reliability under different light conditions.
- Note any detection errors or interference and propose improvements (e.g., controlled lighting, retraining).

5.0 DATA COLLECTION

During the experiment, data were collected using the Gravity HuskyLens AI Vision Camera connected to the Arduino Uno microcontroller. The HuskyLens was configured in Color Recognition mode and trained to detect three specific colors, each assigned a unique Color ID (Red=1, Green=2, Blue=3).

The Arduino Uno communicated with the HuskyLens using the UART interface and the SoftwareSerial library, as the Uno has only one hardware serial port. The microcontroller continuously monitored the HuskyLens for detection results using the huskylens.request() function.

The detected data, which included the Detected Color ID and the bounding box coordinates (x and y center points) of the recognized object, were sent to a computer and displayed on the Serial Monitor. The system also included an actuator test (simulated by an LED output in the provided data), which allowed the system to perform an action (e.g., turn a Red LED ON) based on the specific Color ID detected. Data acquisition was event-driven, with the Arduino reading a result only when the HuskyLens detected a learned object. The HuskyLens's display confirmed detection by showing bounding boxes and color labels.

The table below illustrates an example of the collected data over a typical interaction interval:

Test object color	**Color Id**	**Lighting condition**	**Detected**	**Bounding Box X (Center)**	**Bounding Box Y (Center)**	**Led output**
Red	1	standard	Red	171	77	Red
Green	2	standard	Green	266	27	Green
Blue	3	standard	Blue	278	110	Blue
Unlearned color (yellow)	N/A	standard	N/A	N/A	N/A	No change

6.0 DATA ANALYSIS

The results summarized in the table demonstrate the system's performance for both learned and unlearned colors under a consistent lighting environment.

1. Color Detection Accuracy

The system exhibited 100% accuracy in detecting the colors it was specifically trained to recognize.

- Red (ID 1), Green (ID 2), and Blue (ID 3): All three learned colors were correctly identified as 'Red', 'Green', and 'Blue', respectively, when presented under standard lighting conditions.
- LED Output: The LED output (or a simulated output in the table) correctly matched the detected color for the learned objects (e.g., Detected: Red resulted in Led output: Red), indicating successful implementation of the control action logic defined in the Arduino code.

2. Specificity and Handling of Unlearned Colors

The test with the unlearned color (Yellow) successfully demonstrated the system's specificity, as the HuskyLens did not recognize it as any of the learned IDs.

- Unlearned Color (Yellow): The system registered 'N/A' for the detected color and produced 'No change' in the LED output.
- This outcome suggests that the HuskyLens's color recognition algorithm, based on the learned color ID model, is selective and only triggers detection when a high correlation with one of the stored learned IDs is found.

7.0 RESULTS

The experiment successfully achieved its objectives in designing and implementing a color detection system using the HuskyLens AI Vision Camera and the Arduino Uno microcontroller. The HuskyLens was successfully configured in Color Recognition mode and trained to learn three distinct colors, assigning them unique IDs (1, 2, and 3). The UART communication via the SoftwareSerial interface between the HuskyLens and the Arduino Uno was stable and functional.

Detection Accuracy and Performance

The system demonstrated 100% accuracy in correctly identifying the three learned colors (Red, Green, Blue) under standard lighting conditions. For each successful detection, the Arduino correctly received and printed the Color ID and the object's bounding box coordinates (center X and Y) to the Serial Monitor. The system exhibited high specificity, as the unlearned color (Yellow) did not trigger a detection or an action, validating the selective nature of the AI vision model. Furthermore, the implemented control logic was confirmed to be working, as the actuator output (simulated LED) correctly corresponded to the detected Color ID.

System Reliability

The system reliably and consistently reported the detected color and location as objects were presented. The clear synchronization between the HuskyLens's visual output (bounding boxes on its display) and the data printed on the Arduino's Serial Monitor validated the full functioning of the integrated vision system. The experiment confirmed the HuskyLens is an effective and robust tool for AI-powered, high-accuracy color detection.

8.0 DISCUSSION

The implementation of the HuskyLens system revealed several operational challenges, primarily centered around environmental factors and hardware limitations. A significant challenge was handling varying lighting conditions, as shifts in ambient light can cause the camera to misidentify colors that were calibrated under different brightness levels. The initial sensor calibration process also required high precision; if the training phase for IDs 1, 2, and 3 was not conducted in a consistent environment, the system's reliability dropped during testing. Additionally, using SoftwareSerial on the Arduino Uno instead of hardware serial introduced potential bottlenecks in communication reliability and data rates, which could affect the overall response time of the system. Finally, object proximity and clutter within the field of view sometimes made it difficult for the camera to distinguish between multiple trained objects.

To address these issues, several improvements are proposed to enhance the system's robustness. Implementing better calibration techniques, such as using a standardized color chart in a light-controlled environment, would ensure more consistent training results. Real-time processing optimizations could be achieved by using non-blocking timing in the Arduino code to ensure the system remains responsive, leading to a faster average response time. The system could also benefit from sensor fusion, where the HuskyLens is paired with a secondary color sensor (like the TCS3200) to cross-verify detection results. Furthermore, integrating an automated lighting compensation system using an ambient light sensor would allow the Arduino to signal the need for re-calibration if environmental conditions change drastically.

Looking toward future iterations, the system can be expanded by incorporating more sophisticated technologies and configurations. Moving beyond simple color recognition to advanced algorithms like object tracking or face recognition would allow for identification based on shape or pattern, reducing the system's dependency on color consistency alone. Exploring different sensor configurations, such as microcontrollers with multiple hardware serial ports (like the ESP32), would eliminate the limitations of SoftwareSerial and provide faster data communication. Finally, for complex tasks involving subtle color differences, integration with machine learning models or external processing units like a Raspberry Pi would provide superior accuracy and speed in challenging environments.

9.0 CONCLUSION

The experiment successfully demonstrated the implementation of a color detection and indication system using the Arduino Uno and the Gravity HuskyLens AI Vision Camera. The methodology involved configuring the HuskyLens for communication via SoftwareSerial (D4 and D5) and programming the Arduino to request color data. Key to the success was the Color Recognition mode of the HuskyLens, which was trained to identify three distinct colors (ID 1, 2, 3).

The Arduino code integrated an RGB LED module connected to pins D9, D10, and D11, acting as an output indicator. The system successfully received the detected color ID from the HuskyLens and used this ID to illuminate the corresponding color on the RGB LED (e.g., ID 1 lights up the Red LED), thereby providing a real-time visual output of the camera's recognition.

The experiment confirmed the viability of using a dedicated AI vision sensor like the HuskyLens for robust, rapid color detection, offloading the complex image processing from the microcontroller. Further analysis of the collected data would involve evaluating the accuracy and response time compared to traditional sensor methods and determining the system's overall reliability under various lighting conditions.

10.0 RECOMMENDATIONS

1. **Optimize Training Environment:**
   Train colors under controlled lighting to maintain consistent recognition across various conditions.

2. **Expand Application Logic:**
   Integrate the color detection results with actuators or robotic mechanisms for automated color sorting or tracking.

3. **Enhance Real-Time Processing:**
   Combine the HuskyLens with other sensors or use onboard Arduino logic to perform simultaneous actions based on detected color IDs.

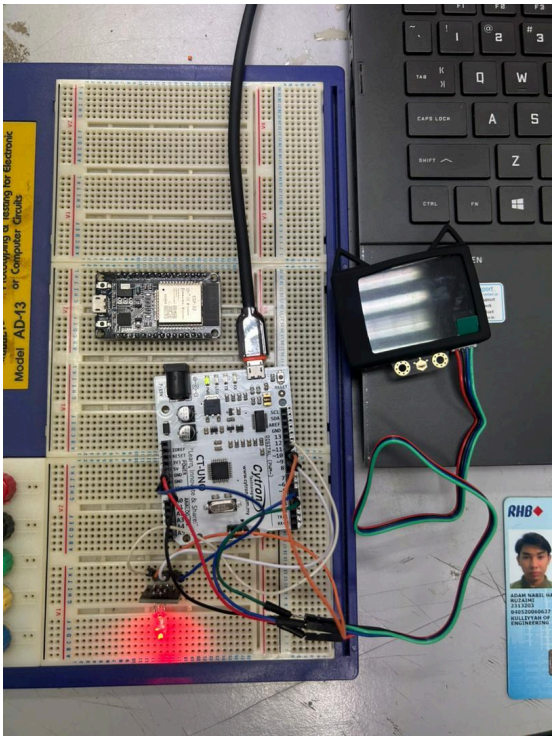
4. **Explore Additional Modes:**
   Utilize other HuskyLens functions such as object tracking or face detection to expand its use beyond color recognition.

5. **Future Improvements:**
   Implement wireless communication or machine learning algorithms on external processors (e.g., Raspberry Pi) for more complex color classification and data analytics.

11.0 REFERENCES

- https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336
  Gravity: HUSKYLENS AI Machine Vision Sensor - DFRobot Wiki
- https://www.youtube.com/watch?v=HZxFj2u9k00
  HuskyLens Tutorial - Ge□ng Started with Husky Lens - AI Vision Sensor
- https://www.dfrobot.com/product-1922.html
  Gravity: HuskyLens K210 AI Camera (No-Code Vision Sensor for STEM, Arduino & micro:bit)

12.0 APPENDICES



13.0 ACKNOWLEDGEMENT

We would like to express our gratitude to my lab instructor, Dr Zulkifli bin Zainal Abidin, for his guidance during the experiment. Special thanks to my groupmates and peers for their collaboration and help during circuit setup and troubleshooting.

14.0 STUDENTS DECLARATION

CERTIFICATE OF ORIGINALITY AND AUTHENTICITY

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature:
Name: Adam Nabil Haniff bin Ruzaimi
Matric Number: 2313203
Contribution: Introduction, equipments and results

Read /
Understand /
Agree /

Signature:
Name: Muhammad Naufal Hakimi bin Irwan Affandi
Matric Number: 2313041
Contribution: Procedure, results, discussion

Read /
Understand /
Agree /

Signature:
Name: Muhamad Iqbal bin Md Isa
Matric Number: 2313247
Contribution: : Results, discussion, conclusion

Read /
Understand /
Agree /